Cyethack Solutions

certin
Enhancing Cyber Security in India

VULNERABILITY ASSESSMENT & PENETRATION TESTING (VAPT) REPORT



L-1 ASSESSMENT REPORT

PREPARED FOR
INDRAPRASTHA INSTITUTE OF INFORMATION TECHNOLOGY DELHI (IIIT-DELHI)

PREPARED BY
CYETHACK SOLUTIONS PVT. LTD.
CERT-IN EMPANELLED CYBERSECURITY COMPANY

VERSION: 1.0

CLIENT VAPT REVIEW/APPROVAL SHEET

	System Owner:				
	Indraprastha Institute of Information Technology Delhi (IIIT-Delhi)			**28-03-2026**	
	Name:			**Date**	
	Assessment Done by:				
	(Cyethack Solutions)			**28-03-2026**	
	Name:			**Date**	
	Cyber Security Reviewer:				
	(Cyethack Solutions)			**28-03-2026**	
	Name:			**Date**	

CONFIDENTIALITY STATEMENT

This document has been prepared exclusively for the client organization named Indraprastha Institute of Information Technology Delhi (IIIT-Delhi). It contains sensitive information and must be treated as CONFIDENTIAL. Distribution to any third party without the explicit written consent of both the client and Cyethack Solutions is strictly prohibited.

The findings are based on the information available during the engagement period (18-03-2026 to 28-03-2026), conducted in accordance with OWASP Testing Guide and **CERT-In** recommendations. This assessment has inherent limitations; the absence of a finding does not imply the absence of a vulnerability.

Disclaimer

The information in this report is provided 'as-is'. Cyethack Solutions makes no warranties regarding completeness or accuracy of the findings and accepts no liability for any loss or damage arising from use of this report.

Document Detail

1. Prepared By-

Identification of Organization that Prepared this Document	
Organization Name	Cyethack Solutions Pvt. Ltd.
Address	7, Vinoba Nagar, Allahabad, Naini,
State	Uttar Pradesh,211008
country	India

2. Prepared For-

Identification of Client	
Organization Name	Indraprastha Institute of Information Technology Delhi (IIIT-Delhi)
Address	Okhla Industrial Estate, Phase III,
State	New Delhi – 110020
Country	India

3. Project Detail-

SN	Title	Vulnerability Assessment & Penetration Testing report
1.	Project Name	Web Application Vulnerability Assessment
2.	Assessment Duration	18/03/2026 – 28/03/2026

4. Document Revision History

SN	Date	Author	Comment
1.	28-03-2026	Cyethack Solutions Pvt. Ltd.	Version 1.0 – Initial L1 Assessment Report

5. Document Distribution List

SN	Name	Organization	Designation	Email ID
1.	(IIIT-Delhi)	IIIT Delhi	Client Representative	tavpriteshsethi@iiitd.ac.in
2.	Project SPOC	Cyethack Solutions Pvt. Ltd.	Security Consultant	support@cyethack.com
3	Management	Cyethack Solutions Pvt. Ltd	Internal Review	support@cyethack.com

Table of Contents

Page #

1. Client VAPT Review/Approval Sheet 01
2. Document Detail 03
3. Executive Summary 05
4. SECURITY ASSESSMENT TOOLS 06
5. SAR Summary 07
6. Detailed Report 08

EXECUTIVE SUMMARY

Cyethack Solutions Private Limited was engaged by Indraprastha Institute of Information Technology Delhi (IIIT-Delhi) to conduct a Black Box **Vulnerability Assessment and Penetration Testing (VAPT)** of the web application https://safeicu.aiims.edu.in.

The assessment was conducted between **18-03-2026** and **28-03-2026** as part of a security assurance and risk management initiative.

The objective of this engagement was to evaluate the security posture of the application and identify vulnerabilities that could impact the confidentiality, integrity, and availability of the system.

The assessment was performed using a combination of manual and automated testing techniques, aligned with industry-recognized best practices, including the **OWASP** Testing Guide and **CERT-In** recommended guidelines.

The key objectives of this assessment were to:

- Identify security vulnerabilities and configuration weaknesses
- Evaluate the effectiveness of existing security controls
- Assess potential risk exposure from identified findings
- Provide actionable recommendations to improve the overall security posture

A total of 13 security findings were identified. The distribution by severity is as follows:

Severity	Count	Recommended Action
Critical	0	Remediate immediately
High	7	Remediate within 7 days
Medium	5	Remediate within 30 days
Low	1	Remediate within 90 days
Info	0	Best practice improvement

*The overall risk posture is rated **HIGH**. Immediate remediation of critical and high-severity findings is required before the application is considered safe for production.*

This **L1 Assessment** Report provides a high-level overview of the identified security findings. Detailed technical analysis, proof-of-concept evidence, and remediation guidance are provided in subsequent sections of the report.

SECURITY ASSESSMENT TOOLS

S. No.	Tool Name	Version
1	Burp Suite Professional	2025.x (Latest Stable)
2	Nmap	7.98 (Latest Stable)
3	CyeNET Suite	Latest Stable Version
4	cURL	8.19.0 (Latest Stable)
5	Bash Scripts	Custom Scripts (Updated)
6	OWASP ZAP	2.16.0 (Latest Stable)
7	Google Chrome Developer Tools	Latest (Auto-Updated)

Detail VAPT L-1 Report
28/03/2026

L-1 SAR SUMMARY

Target Information

https://safeicu.aiims.edu.in

28/03/2026

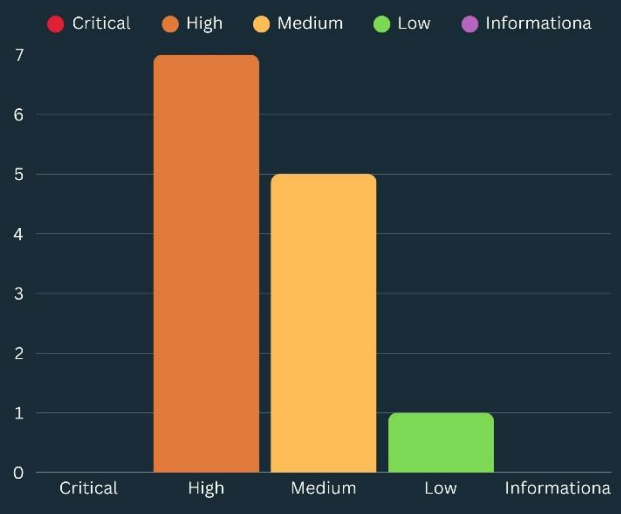
18/03/2026 – 28/03/2026

RISK LEVEL
HIGH

VULNERABILITY

Critical	0
High	7
Medium	5
Low	1
Informational	0

ISSUES FOUND



VULNERABILITY SUMMARY

Total Vulnerabilities Identified in L-1 **13**

This section provides a consolidated overview of all vulnerabilities identified during the L1 Vulnerability Assessment and Penetration Testing (VAPT) engagement.

The identified findings indicate the presence of multiple high-risk vulnerabilities primarily related to security misconfigurations, which may expose sensitive application and infrastructure details.

This section provides an overview of the security findings identified during the Web Application VAPT conducted by Cyethack Solutions for IIIT Delhi on https://safeicu.aiims.edu.in

List of Vulnerable Parameters, Location discovered

The following table provides a consolidated overview of all findings identified during the assessment.

VID	Vulnerability Title	Severity	CWE	Status
1	Server Version Disclosure via HTTP Response Headers	Low	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor	Open
2	Django Debug Mode Enabled Leading to Sensitive Information Disclosure	High	CWE-489 – Active Debug Code	Open
3	Logout Functionality Vulnerable to CSRF (GET Request)	Medium	CWE-352 – Cross-Site Request Forgery (CSRF)	Open
4	Internal IP Address and Port Disclosure	Medium	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor	Open
5	Sensitive Information Disclosure via Improper Error Handling in Login Parameter	High	CWE-209 – Generation of Error Message Containing Sensitive Information	Open
6	Sensitive Infrastructure Information Disclosure via Debug Error (IP, Host, and Headers)	High	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor	Open
7	Django Debug Mode Enabled in Production	High	CWE-489 – Active Debug Code	Open
8	SMTP Configuration and Email Credentials Disclosure via Debug Error	High	CWE-209 – Generation of Error Message Containing Sensitive Information	Open
9	Application Stack and Internal File Path Disclosure via Debug Error	High	CWE-209 – Generation of Error Message Containing Sensitive Information	Open
10	Django Framework and Python Runtime Version Disclosure via Verbose Error Response	High	CWE-209 – Generation of Error Message Containing Sensitive Information	Open
11	Outdated Django Version (4.1.5) with Known Security Vulnerabilities	Medium	CWE-1104 – Use of Unmaintained Third Party Components	Open
12	Outdated nginx Version (1.24.0) with Known Security Risks	Medium	CWE-1104 – Use of Unmaintained Third Party Components	Open
13	Missing Secure Flag on Session Cookies	Medium	CWE-614 – Sensitive Cookie in HTTPS Session Without 'Secure' Attribute	Open

VULNERABILITY 1

1. Vulnerable Asset Detail

Finding 1 — Server Version Disclosure via HTTP Response Headers

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/dua/
Vulnerability Title	Server Version Disclosure via HTTP Response Headers
Severity	**Low**
CWE	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor
Reference	https://cwe.mitre.org/data/definitions/200.html

2. Detailed Observation / Proof of Concept

During testing of the endpoint:

https://safeicu.aiims.edu.in/dua/

the application intermittently returned a 502 Bad Gateway error page, which disclosed backend server information.

Additionally, the HTTP response headers explicitly reveal the server version:

Server: nginx/1.24.0 (Ubuntu)

Observation:

- The error page displays:
- Web server: nginx
- Version: 1.24.0
- Operating System: Ubuntu
- The same information is also exposed in HTTP response headers.

Request:

GET /dua/ HTTP/2

Host: safeicu.aiims.edu.in

Response:

HTTP/2 302 Found

Server: nginx/1.24.0 (Ubuntu)

Screenshot Reference:

- Screenshot 1: Error page showing nginx version disclosure
- Screenshot 2: Response header showing server version

Technical Explanation:

The application server is configured to include detailed version information in HTTP headers and default error pages. This behavior allows attackers to identify the exact server version and underlying technology stack.

3. Impact

The disclosure of server version information can assist attackers in reconnaissance and targeted attacks.

Potential risks include:

- Identification of specific server version (nginx 1.24.0)
- Mapping known vulnerabilities associated with the version
- Increased effectiveness of automated attacks and scanners
- Technology stack fingerprinting

While this issue does not directly lead to exploitation, it increases the overall attack surface.

4. Remediation & References

Remediation:

It is recommended to disable server version disclosure in both HTTP headers and error responses.

- Disable version tokens in nginx configuration:
  server_tokens off;

- Configure custom error pages to prevent information leakage
- Remove or mask the "Server" response header where possible
- Follow secure server hardening practices

References:

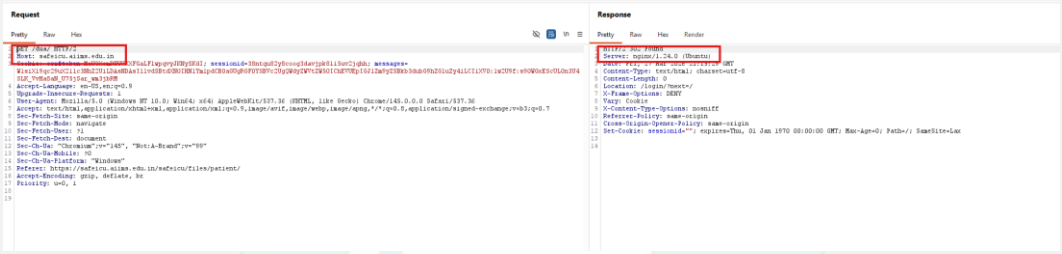
- CWE-200: https://cwe.mitre.org/data/definitions/200.html
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
- Nginx Documentation: https://nginx.org/en/docs/http/ngx_http_core_module.html#server_tokens

Evidence / Screenshots

Figure: V1-Version Disclosure Ubuntu.png



Figure: V1.1-Version Disclosure Ubuntu.png



VULNERABILITY 2

Finding 2 — Django Debug Mode Enabled Leading to Sensitive Information Disclosure

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/administrator
Vulnerability Title	Django Debug Mode Enabled Leading to Sensitive Information Disclosure
Severity	**High**
CWE	CWE-489 – Active Debug Code
Reference	https://cwe.mitre.org/data/definitions/489.html

2. Detailed Observation / Proof of Concept

During testing, access the following endpoint:

https://safeicu.aiims.edu.in/administrator

resulted in a verbose error page indicating that the application is running with DEBUG = True in the Django configuration.

The response displays detailed internal information including:

- URL routing patterns
- Internal application endpoints
- Framework behavior and configuration hints
- Debug message explicitly stating debug mode is enabled

Observation:

The error page contains the following message:

"You're seeing this error because you have DEBUG = True in your Django settings file. Change that to False, and Django will display a standard 404 page."

Additionally, multiple internal URL patterns were exposed, such as:

- /login/
- /register/
- /safeicu/v1.0/patients/
- /safeicu/files/<str:table_code>/
- /password_reset/

Screenshot Reference:

- V2- DEBUG TRUE Disclose Internal Path.png

Technical Explanation:

Django applications running with DEBUG = True expose detailed error messages and internal configurations. This includes URL mappings, application structure, and debugging information intended only for development environments.

Such information should never be exposed in production as it significantly aids attackers in understanding the internal architecture of the application.

3. Impact

This vulnerability poses a significant security risk as it exposes sensitive internal details of the application.

Potential risks include:

- Disclosure of internal application structure and endpoints
- Exposure of hidden or unlinked functionalities
- Increased attack surface for targeted exploitation
- Facilitation of further attacks such as IDOR, authentication bypass, or API abuse
- Leakage of framework and configuration details

If combined with other vulnerabilities, this information can be leveraged to perform more advanced and targeted attacks.

4. Remediation & References

Remediation:

It is strongly recommended to disable debug mode in production environments.

- Update Django settings:

DEBUG = False

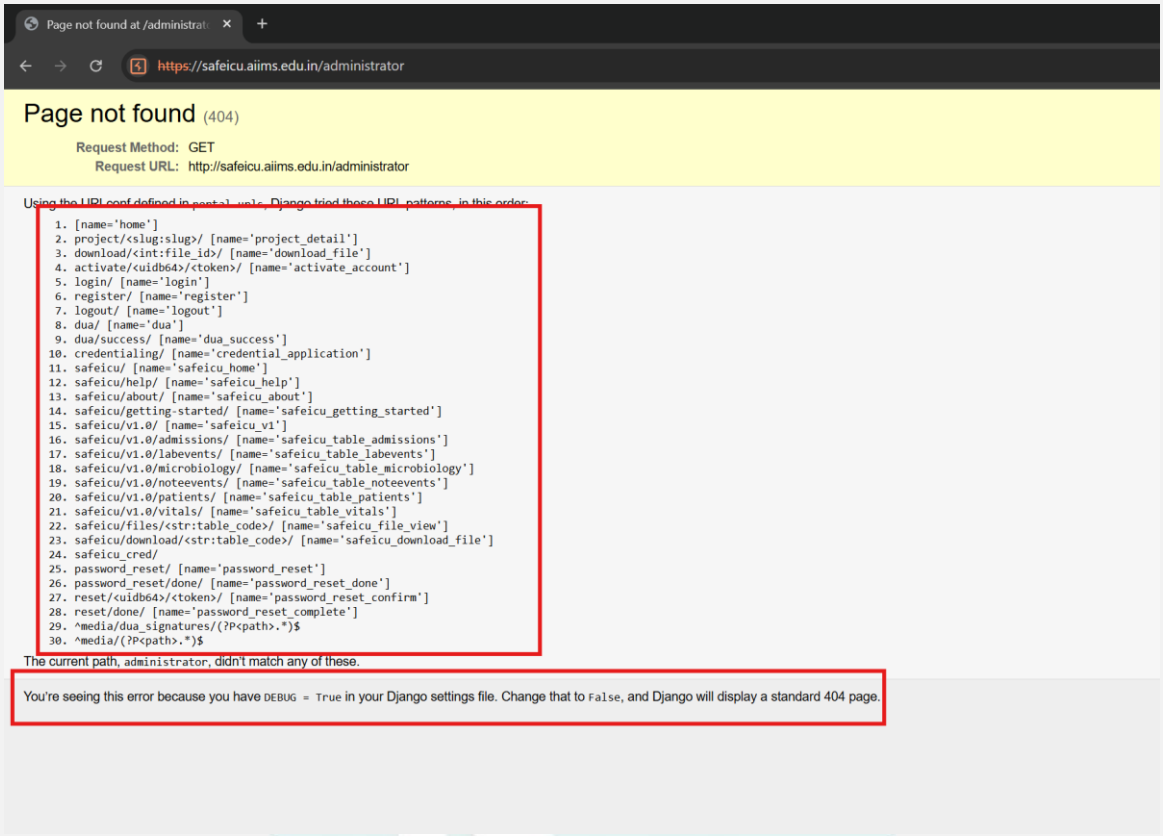
- Configure proper error handling and custom error pages
- Ensure sensitive information is not exposed in any error responses
- Implement centralized logging instead of exposing errors to users
- Restrict access to administrative and internal endpoints

References:

- CWE-489: https://cwe.mitre.org/data/definitions/489.html
- OWASP Information Disclosure: https://owasp.org/www-community/attacks/Information_exposure_through_error_messages
- Django Security Documentation: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V2- DEBUG TRUE Disclose Internal Path.png



VULNERABILITY 3

Finding 3 — Logout Functionality Vulnerable to CSRF (GET Request)

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/logout/
Vulnerability Title	Logout Functionality Vulnerable to CSRF (GET Request)
Severity	**Medium**
CWE	CWE-352 – Cross-Site Request Forgery (CSRF)
Reference	https://cwe.mitre.org/data/definitions/352.html

2. Detailed Observation / Proof of Concept

During testing, it was observed that the logout functionality is triggered via a GET request without requiring any CSRF protection.

Request:

GET /logout/ HTTP/2

Host: safeicu.aiims.edu.in

Response:

HTTP/2 302 Found

Location: /project/safeicu/

Observation:

- Logout action is performed using a simple GET request
- No CSRF token validation is enforced
- The request can be triggered externally (e.g., via malicious link or embedded image)

Attack Scenario:

An attacker can craft a malicious link such as:

https://safeicu.aiims.edu.in/logout/

If a logged-in user visits this link (via email, phishing page, or embedded resource), their session will be terminated without their consent.

Screenshot Reference:

- V3-Logout CSRF.png

Technical Explanation:

Sensitive actions such as logout should not be performed via GET requests, as they are inherently vulnerable to CSRF attacks. Since browsers automatically include session cookies in requests, an attacker can exploit this behavior to force unintended actions.

3. Impact

This vulnerability allows attackers to forcefully log out authenticated users without their interaction.

Potential risks include:

- Session disruption for users
- Potential abuse in phishing or social engineering scenarios
- Reduced user trust and experience
- Can be combined with other attacks (e.g., login CSRF or session fixation scenarios)

Although this does not directly compromise data, it impacts application security posture and user session integrity.

4. Remediation & References

Remediation:

- Change logout functionality to use POST request only
- Implement CSRF protection:
- Require valid CSRF token for logout action
- Validate token on server-side
- Avoid performing sensitive actions via GET requests
- Implement SameSite cookie protection where applicable

References:

- CWE-352: https://cwe.mitre.org/data/definitions/352.html
- OWASP CSRF Guide: https://owasp.org/www-community/attacks/csrf
- OWASP Top 10 (A01:2021): https://owasp.org/Top10/

Evidence / Screenshots

Figure: V3-Logout CSRF.png



VULNERABILITY 4

Finding 4 — Internal IP Address and Port Disclosure

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/project/safeicu/
Vulnerability Title	Internal IP Address and Port Disclosure
Severity	**Medium**
CWE	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor
Reference	https://cwe.mitre.org/data/definitions/200.html

2. Detailed Observation / Proof of Concept

During testing of the endpoint:

https://safeicu.aiims.edu.in/project/safeicu/

it was observed that the application response contains a reference to an internal IP address and port.

Observation:

The response includes the following internal URL:

http://192.168.13.118:8003/safeicu/

This indicates that the backend application is hosted on an internal network and is unintentionally exposing internal infrastructure details.

Request:

GET /project/safeicu/ HTTP/2

Host: safeicu.aiims.edu.in

Screenshot Reference:

- V4- Internal IP & Port Disclosure.png

Technical Explanation:

The application embeds internal service URLs directly in responses. These URLs reveal private IP addresses and service ports that are not intended to be publicly accessible.

Such disclosures provide attackers with insight into internal network architecture, which can be leveraged for further attacks such as SSRF, lateral movement, or targeted exploitation of internal services.

3. Impact

This vulnerability exposes sensitive internal infrastructure details.

Potential risks include:

- Disclosure of internal IP addressing scheme
- Identification of backend services and ports
- Increased risk of Server-Side Request Forgery (SSRF) exploitation
- Facilitation of lateral movement in case of further compromise
- Improved attacker reconnaissance capabilities

While not directly exploitable on its own, this information significantly aids attackers in planning targeted attacks.

4. Remediation & References

Remediation:

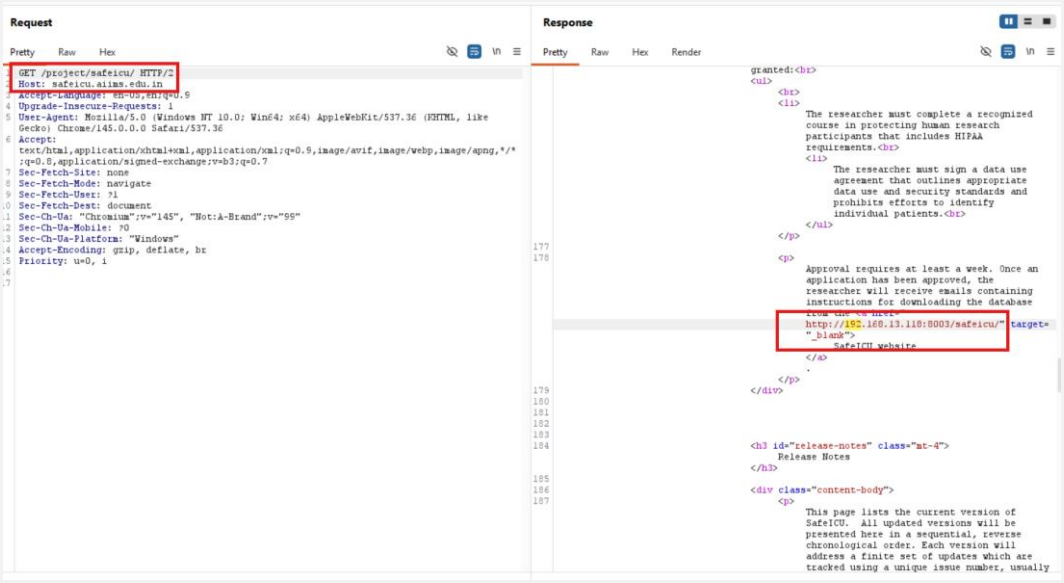
- Avoid exposing internal IP addresses in application responses
- Replace internal URLs with publicly accessible domain names
- Use reverse proxy or API gateway to mask backend infrastructure
- Perform response filtering to remove sensitive internal details
- Conduct regular security reviews to identify unintended data exposure

References:

- CWE-200: https://cwe.mitre.org/data/definitions/200.html
- OWASP Information Disclosure: https://owasp.org/www-community/attacks/Information_exposure_through_error_messages
- OWASP Top 10 (A05:2021 - Security Misconfiguration): https://owasp.org/Top10/

Evidence / Screenshots

Figure: V4- Internal IP & Port Disclosure.png


Request
Pretty Raw Hex
GET /project/safeicu/ HTTP/2
Host: safeicu.aiims.edu.in
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Response
Pretty Raw Hex Render
granted:

The researcher must complete a recognized course in protecting human research participants that includes HIPAA requirements.

The researcher must sign a data use agreement that outlines appropriate data use and security standards and prohibits efforts to identify individual patients.

Approval requires at least a week. Once an application has been approved, the researcher will receive emails containing instructions for downloading the database
http://192.168.13.118:8003/safeicu/" target="_blank">
SafeICU website

.

<h3 id="release-notes" class="mt-4">
Release Notes
</h3>

This page lists the current version of SafeICU. All updated versions will be presented here in a sequential, reverse chronological order. Each version will address a finite set of updates which are tracked using a unique issue number, usually

VULNERABILITY 5

Finding 5 — Sensitive Information Disclosure via Improper Error Handling in Login Parameter

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Sensitive Information Disclosure via Improper Error Handling in Login Parameter
Severity	**High**
CWE	CWE-209 – Generation of Error Message Containing Sensitive Information
Reference	https://cwe.mitre.org/data/definitions/209.html

2. Detailed Observation / Proof of Concept

During testing of the login functionality, a specially crafted payload was injected into the next parameter:

Payload used:

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request:

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation:

- The payload caused the application to throw an error
- Due to DEBUG mode being enabled, a detailed error page was displayed
- The response exposed sensitive environment and configuration details

Sensitive Information Exposed:

- Database Name: safeicu_db
- Database Host: db
- Database Port: 5432
- Database User: safeicu_user
- Application PATH variables
- Internal environment configuration

Screenshot Reference:

- V5- Databse disclosure in the error through.png

Technical Explanation:

The application fails to properly handle malformed input in the next parameter, resulting in an unhandled exception. Due to DEBUG mode being enabled, the application returns verbose error messages containing sensitive configuration data.

This includes environment variables and database connection details, which should never be exposed to end users.

3. Impact

This vulnerability leads to exposure of highly sensitive backend configuration details.

Potential risks include:

- Disclosure of database credentials and structure
- Exposure of internal environment variables
- Increased likelihood of database-targeted attacks
- Facilitation of lateral movement within infrastructure
- Significant advantage to attackers during reconnaissance

If combined with other vulnerabilities, this could lead to full system compromise.

4. Remediation & References

Remediation:

- Disable debug mode in production:

  DEBUG = False

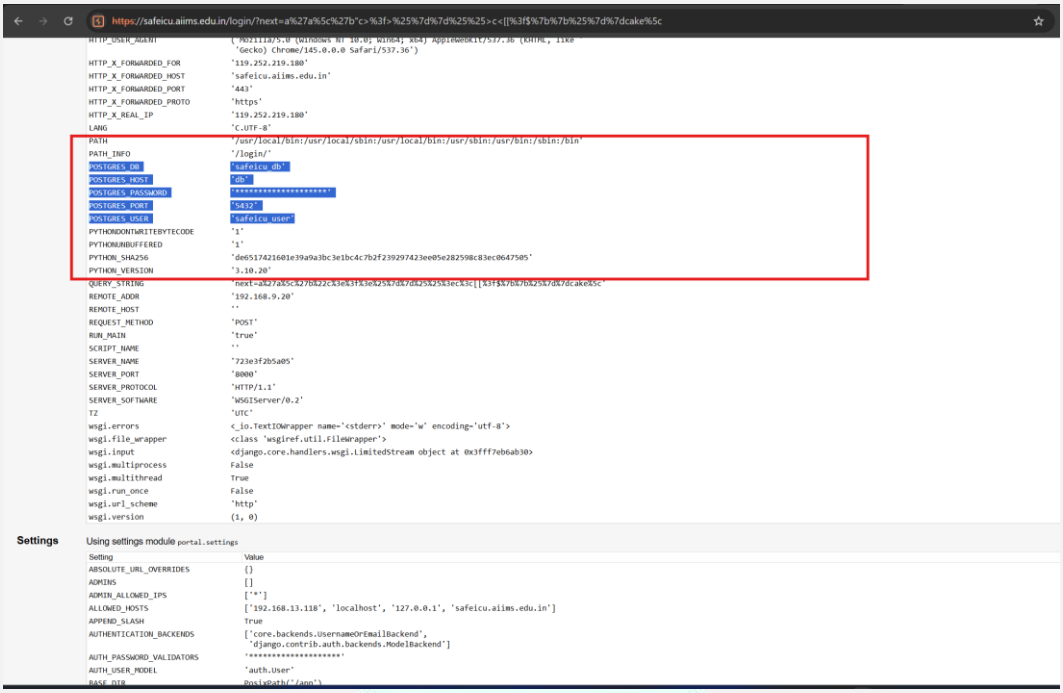
- Implement proper error handling to avoid exposing stack traces and environment details
- Validate and sanitize all user-controlled inputs, especially URL parameters like next
- Avoid reflecting or processing untrusted input without validation
- Use generic error messages for users and log detailed errors securely on the server side

References:

- CWE-209: https://cwe.mitre.org/data/definitions/209.html
- OWASP Error Handling: https://owasp.org/www-community/Improper_Error_Handling
- Django Security Best Practices: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V5- Databse disclosure in the error through.png


https://safeicu.aiims.edu.in/login/?next=a%27a%5c%27b"c>%3f>%25%7d%7d%25%25><([%3f$%7b%7b%25%7d%7dcake%5c
HTTP_USER_AGENT
('Gecko) Chrome/145.0.0.0 Safari/537.36')
HTTP_X_FORWARDED_FOR '119.252.219.180'
HTTP_X_FORWARDED_HOST 'safeicu.aiims.edu.in'
HTTP_X_FORWARDED_PORT '443'
HTTP_X_FORWARDED_PROTO 'https'
HTTP_X_REAL_IP '119.252.219.180'
LANG 'C.UTF-8'
PATH '/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
PATH_INFO '/login/'
POSTGRES_DB 'safeicu_db'
POSTGRES_HOST 'db'
POSTGRES_PASSWORD '**********'
POSTGRES_PORT '5432'
POSTGRES_USER 'safeicu_user'
PYTHONDONTWRITEBYTECODE '1'
PYTHONUNBUFFERED '1'
PYTHON_SHA256 'de6517421601e39a9a3bc3e1bc4c7b2f239297423ee05e282598c83ec0647505'
PYTHON_VERSION '3.10.20'
QUERY_STRING 'next=a%27a%5c%27b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c|%3f$%7b%7b%25%7d%7dcake%5c'
REMOTE_ADDR '192.168.9.20'
REMOTE_HOST ''
REQUEST_METHOD 'POST'
RUN_MAIN 'true'
SCRIPT_NAME ''
SERVER_NAME '723e3f2b5a05'
SERVER_PORT '8000'
SERVER_PROTOCOL 'HTTP/1.1'
SERVER_SOFTWARE 'WSGIServer/0.2'
TZ 'UTC'
wsgi.errors <_io.TextIOWrapper name='<stderr>' mode='w' encoding='utf-8'>
wsgi.file_wrapper <class 'wsgiref.util.FileWrapper'>
wsgi.input <django.core.handlers.wsgi.LimitedStream object at 0x3fff7eb6ab30>
wsgi.multiprocess False
wsgi.multithread True
wsgi.run_once False
wsgi.url_scheme 'http'
wsgi.version (1, 0)
Settings
Using settings module portal.settings
Setting Value
ABSOLUTE_URL_OVERRIDES {}
ADMINS []
ADMIN_ALLOWED_IPS ['*']
ALLOWED_HOSTS ['192.168.13.118', 'localhost', '127.0.0.1', 'safeicu.aiims.edu.in']
APPEND_SLASH True
AUTHENTICATION_BACKENDS ['core.backends.UsernameOrEmailBackend',
'django.contrib.auth.backends.ModelBackend']
AUTH_PASSWORD_VALIDATORS '**********'
AUTH_USER_MODEL 'auth.User'

VULNERABILITY 6

Finding 6 — Sensitive Infrastructure Information Disclosure via Debug Error (IP, Host, and Headers)

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Sensitive Infrastructure Information Disclosure via Debug Error (IP, Host, and Headers)
Severity	**High**
CWE	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor
Reference	https://cwe.mitre.org/data/definitions/200.html

2. Detailed Observation / Proof of Concept

During testing of the login functionality, a crafted payload was injected into the next parameter:

Payload used:

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request:

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation:

- The payload triggered an application error
- Due to DEBUG mode enabled, detailed system information was exposed
- The error page disclosed network and infrastructure-related details

Sensitive Information Exposed:

- Internal Allowed Hosts:
- 192.168.13.118
- localhost
- 127.0.0.1
- safeicu.aiims.edu.in
- Client IP Information:
- HTTP_X_FORWARDED_FOR: 14.139.38.215

- HTTP_X_REAL_IP: 14.139.38.215
- Server Configuration Details:
- HTTP_X_FORWARDED_HOST
- HTTP_X_FORWARDED_PORT: 443
- HTTP_X_FORWARDED_PROTO: https

Screenshot Reference:

- v6- IP port and host name disclusure.png

Technical Explanation:

The application exposes sensitive infrastructure details due to improper error handling combined with DEBUG mode enabled. The error response reveals internal host configurations, IP addresses, and request header data.

Such information should remain confidential and only be accessible internally, as it provides attackers with a deeper understanding of the application's deployment environment.

3. Impact

This vulnerability exposes critical infrastructure and network-level information.

Potential risks include:

- Disclosure of internal IP addresses and host configurations
- Exposure of reverse proxy or load balancer headers
- Improved attacker reconnaissance capabilities
- Facilitation of targeted attacks on internal systems
- Increased likelihood of SSRF, header manipulation, or network-based attacks

When combined with other vulnerabilities, this can significantly increase the risk of full system compromise.

4. Remediation & References

Remediation:

- Disable debug mode in production:
  DEBUG = False
- Prevent exposure of sensitive headers and internal configuration in error responses
- Implement secure error handling with generic user-facing messages
- Restrict and sanitize sensitive headers such as:
- X-Forwarded-For
- X-Real-IP

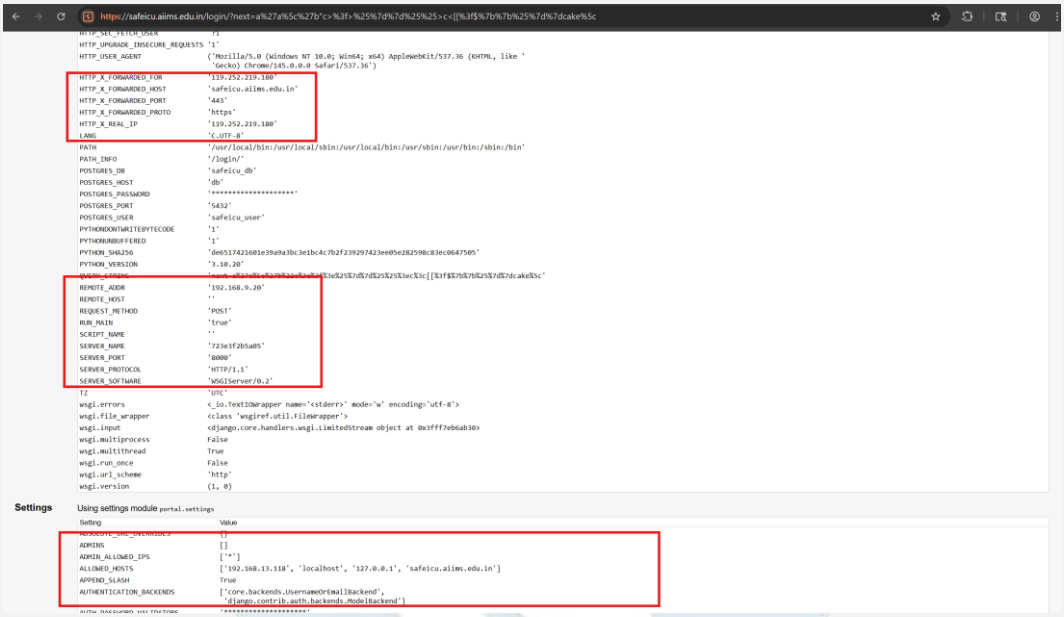
- Use proper server and proxy configurations to avoid leaking internal infrastructure details

References:

- CWE-200: https://cwe.mitre.org/data/definitions/200.html
- OWASP Information Disclosure: https://owasp.org/www-community/attacks/Information_exposure_through_error_messages
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/

Evidence / Screenshots

Figure: v6- IP port and host name disclusure.png



VULNERABILITY 7

Finding 7 — Django Debug Mode Enabled in Production

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Django Debug Mode Enabled in Production
Severity	**High**
CWE	CWE-489 – Active Debug Code
Reference	https://cwe.mitre.org/data/definitions/489.html

2. Detailed Observation / Proof of Concept

During testing, a crafted payload was injected into the next parameter of the login functionality:

Payload used:

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request:

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation:

- The application returned a verbose Django error page
- The response clearly indicates that DEBUG = True
- The debug page exposes internal configuration and application settings

Evidence:

The following configuration was visible in the response:

- DEBUG = True
- DEBUG_PROPAGATE_EXCEPTIONS = False

Screenshot Reference:

- V7 Django Debug true enabled.png

Technical Explanation:

Django applications with DEBUG = True enabled expose detailed debugging information, including stack traces, environment variables, settings, and internal configurations.

This setting is intended strictly for development environments and should never be enabled in production systems.

3. Impact

Enabling debug mode in a production environment significantly increases the risk of sensitive information disclosure.

Potential risks include:

- Exposure of application configuration and settings
- Disclosure of environment variables and secrets
- Leakage of database and infrastructure details
- Increased attack surface for exploitation
- Enables attackers to gather detailed intelligence about the system

This vulnerability acts as a root cause for multiple other information disclosure issues identified in the application.

4. Remediation & References

Remediation:

- Disable debug mode in production environments:

  DEBUG = False

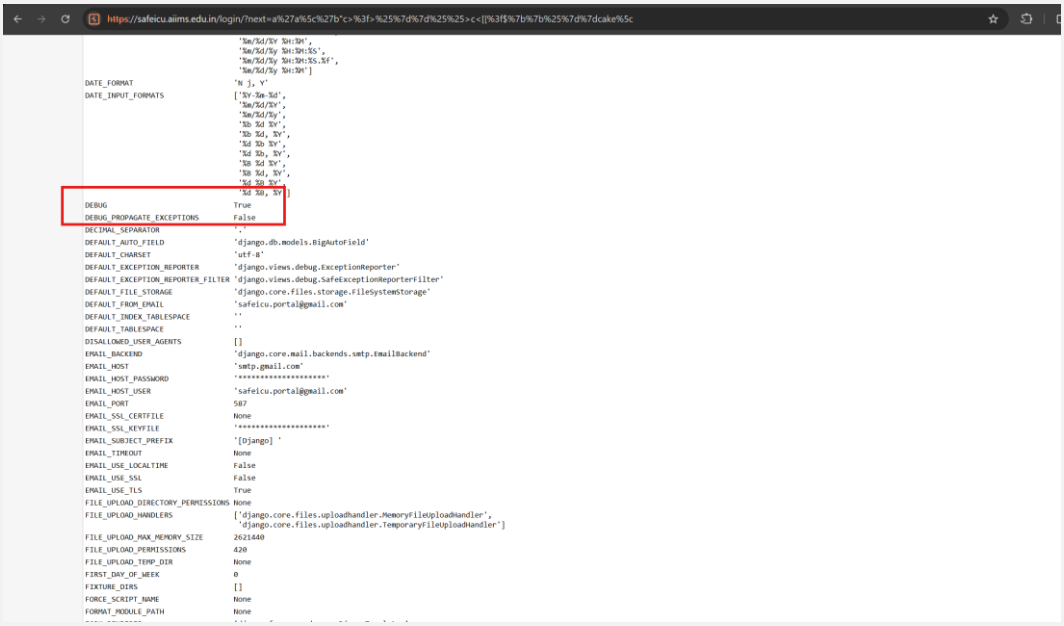
- Use environment-based configuration to ensure debug mode is never enabled in production
- Configure proper logging mechanisms instead of exposing errors to users
- Implement custom error pages to prevent leakage of sensitive information
- Restrict access to debugging and administrative interfaces

References:

- CWE-489: https://cwe.mitre.org/data/definitions/489.html
- Django Security Documentation: https://docs.djangoproject.com/en/stable/topics/security/
- OWASP Security Misconfiguration: https://owasp.org/www-project-top-ten/

Evidence / Screenshots

Figure: V7 Django Debug true enabled.png


https://safeicu.aiims.edu.in/login/?next=a%27a%5c%27b"c>%3f>%25%7d%7d%25%25><<[[%3f|%7b%7b%25%7d%7dcake%5c
'%m/%d/%Y %H:%M',
'%m/%d/%y %H:%M:%S',
'%m/%d/%y %H:%M:%S.%f',
'%m/%d/%y %H:%M']
DATE_FORMAT 'N j, Y'
DATE_INPUT_FORMATS ['%Y-%m-%d',
'%m/%d/%Y',
'%m/%d/%y',
'%b %d %Y',
'%b %d, %Y',
'%d %b %Y',
'%d %b, %Y',
'%B %d %Y',
'%B %d, %Y',
'%d %B %Y',
'%d %B, %Y']
DEBUG True
DEBUG_PROPAGATE_EXCEPTIONS False
DECIMAL_SEPARATOR '.'
DEFAULT_AUTO_FIELD 'django.db.models.BigAutoField'
DEFAULT_CHARSET 'utf-8'
DEFAULT_EXCEPTION_REPORTER 'django.views.debug.ExceptionReporter'
DEFAULT_EXCEPTION_REPORTER_FILTER 'django.views.debug.SafeExceptionReporterFilter'
DEFAULT_FILE_STORAGE 'django.core.files.storage.FileSystemStorage'
DEFAULT_FROM_EMAIL 'safeicu.portal@gmail.com'
DEFAULT_INDEX_TABLESPACE ''
DEFAULT_TABLESPACE ''
DISALLOWED_USER_AGENTS []
EMAIL_BACKEND 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST 'smtp.gmail.com'
EMAIL_HOST_PASSWORD '********************'
EMAIL_HOST_USER 'safeicu.portal@gmail.com'
EMAIL_PORT 587
EMAIL_SSL_CERTFILE None
EMAIL_SSL_KEYFILE '********************'
EMAIL_SUBJECT_PREFIX '[Django] '
EMAIL_TIMEOUT None
EMAIL_USE_LOCALTIME False
EMAIL_USE_SSL False
EMAIL_USE_TLS True
FILE_UPLOAD_DIRECTORY_PERMISSIONS None
FILE_UPLOAD_HANDLERS ['django.core.files.uploadhandler.MemoryFileUploadHandler',
'django.core.files.uploadhandler.TemporaryFileUploadHandler']
FILE_UPLOAD_MAX_MEMORY_SIZE 2621440
FILE_UPLOAD_PERMISSIONS 420
FILE_UPLOAD_TEMP_DIR None
FIRST_DAY_OF_WEEK 0
FIXTURE_DIRS []
FORCE_SCRIPT_NAME None
FORMAT_MODULE_PATH None

VULNERABILITY 8

Finding 8 — SMTP Configuration and Email Credentials Disclosure via Debug Error

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	SMTP Configuration and Email Credentials Disclosure via Debug Error
Severity	**High**
CWE	CWE-209 – Generation of Error Message Containing Sensitive Information
Reference	https://cwe.mitre.org/data/definitions/209.html

2. Detailed Observation / Proof of Concept

During testing, a crafted payload was injected into the next parameter of the login functionality:

Payload used:

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request:

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation:

- The payload triggered an application error
- Due to DEBUG mode enabled, a verbose error page was displayed
- The response exposed sensitive email and SMTP configuration details

Sensitive Information Exposed:

- Default Email:
- safeicu.portal@gmail.com

- SMTP Configuration:
- EMAIL_BACKEND: django.core.mail.backends.smtp.EmailBackend
- EMAIL_HOST: smtp.gmail.com
- EMAIL_PORT: 587
- EMAIL_HOST_USER: safeicu.portal@gmail.com

- Additional Configuration:
- EMAIL_SSL_CERTFILE: None

Screenshot Reference:

- V8 Email ID and condifguration disclosed.png

Technical Explanation:

The application exposes email configuration details due to improper error handling combined with DEBUG mode enabled. The error page reveals SMTP server configuration and associated email account information.

Even if passwords are masked, disclosure of such configuration details significantly weakens the security posture and may aid attackers in targeted phishing or credential-based attacks.

3. Impact

This vulnerability exposes sensitive email infrastructure details.

Potential risks include:

- Disclosure of application email account (safeicu.portal@gmail.com)
- Increased risk of phishing or social engineering attacks
- Potential targeting of SMTP services for abuse or brute force attempts
- Exposure of backend communication mechanisms
- Reduced confidentiality of internal configuration

When combined with other vulnerabilities, this could lead to further compromise of communication channels.

4. Remediation & References

Remediation:

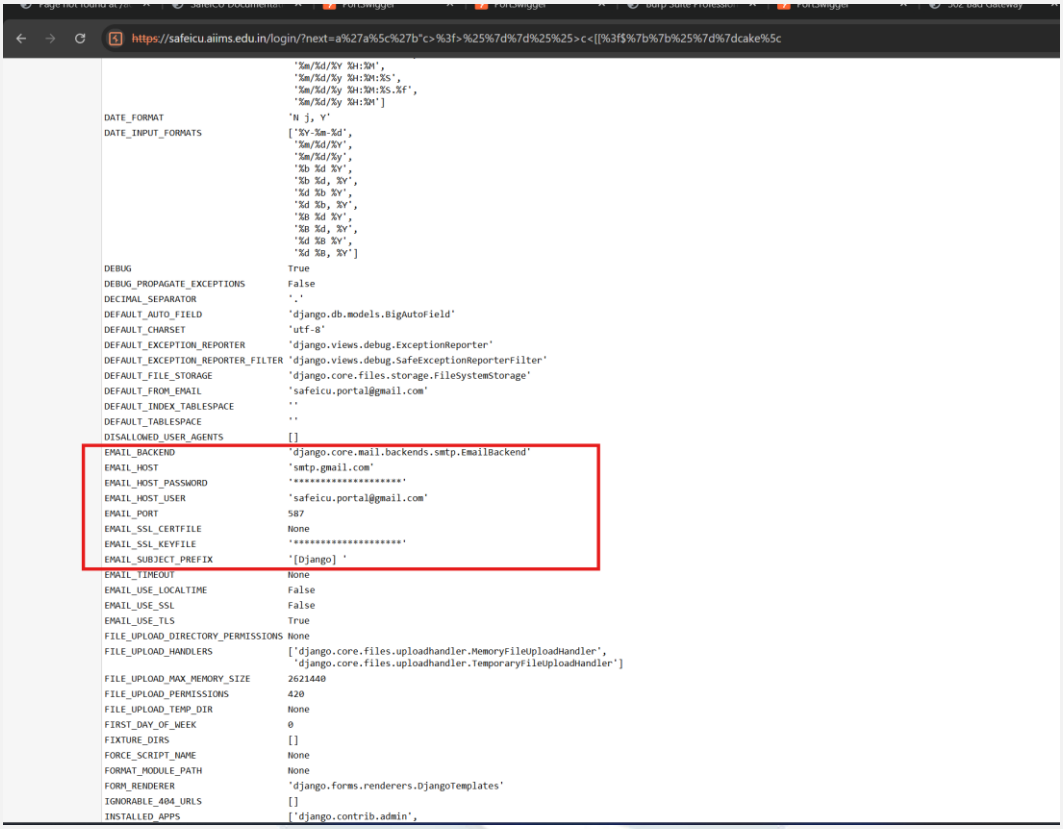
- Disable debug mode in production:
  DEBUG = False
- Ensure sensitive configuration values are never exposed in error responses
- Store sensitive credentials securely using environment variables or secret management systems
- Implement proper error handling with generic user-facing messages
- Restrict access to configuration details and logs

References:

- CWE-209: https://cwe.mitre.org/data/definitions/209.html
- OWASP Sensitive Data Exposure: https://owasp.org/www-project-top-ten/
- Django Security Best Practices: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V8 Email ID and condifguration disclosed.png



VULNERABILITY 9

Finding 9 — Application Stack and Internal File Path Disclosure via Debug Error

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Application Stack and Internal File Path Disclosure via Debug Error
Severity	**High**
CWE	CWE-209 – Generation of Error Message Containing Sensitive Information
Reference	https://cwe.mitre.org/data/definitions/209.html

2. Detailed Observation / Proof of Concept

During testing, a crafted payload was injected into the next parameter of the login functionality:

Payload used:

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request:

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation:

- The payload triggered an application error
- Due to DEBUG mode enabled, a detailed Django error page was displayed
- The response exposed internal application stack and file system details

Sensitive Information Exposed:

- Python Executable:
- /usr/local/bin/python
- Python Version:
- 3.10.20
- Application Paths:
- /app
- /usr/local/lib/python310.zip
- /usr/local/lib/python3.10
- /usr/local/lib/python3.10/lib-dynload

- /usr/local/lib/python3.10/site-packages

- Application Code Reference:
- core.views.login_view

Screenshot Reference:

- V9 error through internal path and files and python version.png

Technical Explanation:

The application exposes internal runtime and file system information due to improper error handling combined with DEBUG mode enabled. The debug page reveals Python runtime details, library paths, and application structure.

This information can be used by attackers to identify framework versions, locate sensitive files, and craft targeted exploits.

3. Impact

This vulnerability exposes critical details about the application environment and runtime.

Potential risks include:

- Disclosure of backend technology stack (Python, Django)
- Exposure of internal file system structure
- Identification of application entry points and code paths
- Increased effectiveness of targeted attacks and exploitation
- Facilitation of vulnerability chaining

Such information significantly enhances attacker reconnaissance capabilities.

4. Remediation & References

Remediation:

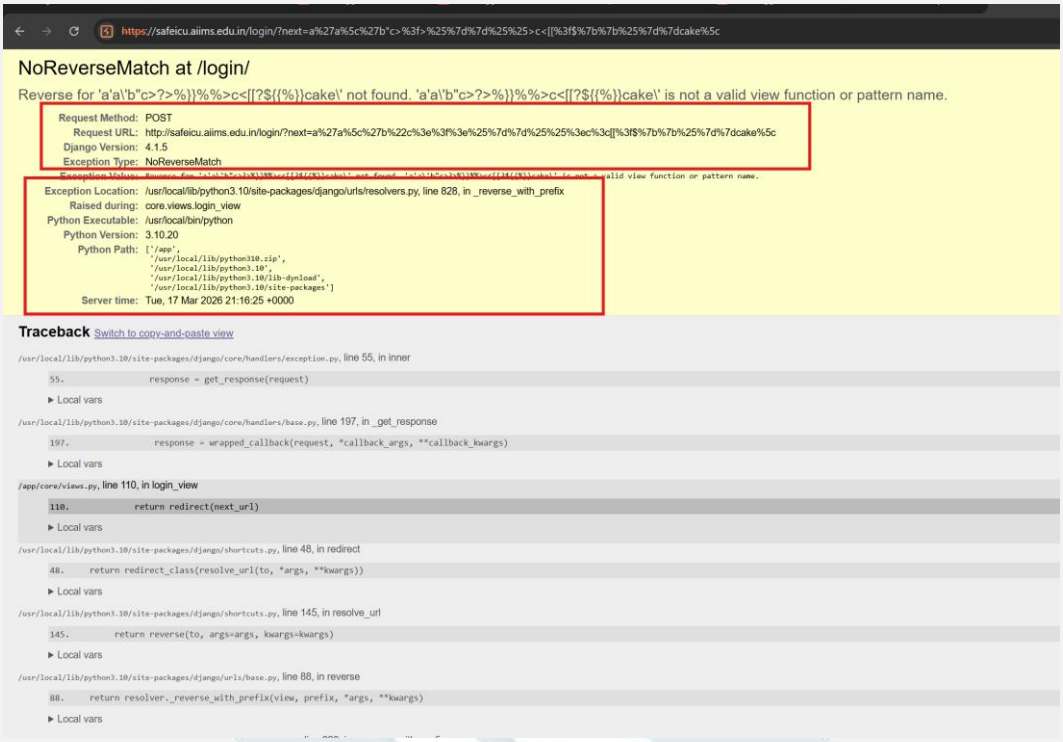
- Disable debug mode in production:
  DEBUG = False
- Prevent exposure of stack traces and internal paths in error responses
- Implement secure error handling with generic user-facing messages
- Restrict access to debugging information
- Ensure proper logging mechanisms are used instead of exposing errors

References:

- CWE-209: https://cwe.mitre.org/data/definitions/209.html
- OWASP Error Handling: https://owasp.org/www-community/Improper_Error_Handling
- Django Security Documentation: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V9 error through internal path and files and python version.png



VULNERABILITY 10

Finding 10 — Django Framework and Python Runtime Version Disclosure via Verbose Error Response

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Django Framework and Python Runtime Version Disclosure via Verbose Error Response
Severity	**High**
CWE	CWE-209 – Generation of Error Message Containing Sensitive Information
Reference	https://cwe.mitre.org/data/definitions/209.html

2. Detailed Observation / Proof of Concept

During testing of the login functionality, a crafted payload was injected into the next parameter to observe the application's error handling behavior.

Payload Used

a'a\'b"c>?>%}}%%>c<[[?${{%}}cake\

Request

POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f$%7b%7b%25%7d%7dcake%5c HTTP/2

Host: safeicu.aiims.edu.in

Observation

It was observed that the malformed next parameter caused the application to throw a 500 Internal Server Error. Due to verbose error handling and debug information exposure, the response disclosed sensitive framework and runtime details.

The error page revealed the following information:

- Request Method: POST
- Request URL: http://safeicu.aiims.edu.in/login/?next=...
- Django Version: 4.1.5
- Exception Type: NoReverseMatch
- Raised during: core.views.login_view

- Python Executable: /usr/local/bin/python
- Python Version: 3.10.20
- Python Path:
- /app
- /usr/local/lib/python310.zip
- /usr/local/lib/python3.10
- /usr/local/lib/python3.10/lib-dynload
- /usr/local/lib/python3.10/site-packages

Screenshot Reference

- V10 500 error through the INternal Path and Python version.png
- V10.1 error through the INternal Path and Python version.png

Technical Explanation

The application processes the user-controlled next parameter without safely handling malformed values. When an invalid value is submitted, the application throws an exception and returns a verbose error page instead of a generic error response.

This exposes detailed information about the underlying framework, application code path, Python runtime, and installed environment structure. Such details are valuable to attackers during reconnaissance and exploit development.

3. Impact

This vulnerability discloses sensitive application stack and runtime information to an unauthenticated or low-privileged user.

Potential risks include:

- Identification of exact Django version in use
- Identification of exact Python runtime version
- Disclosure of internal code reference such as core.views.login_view
- Exposure of server-side file paths and library structure
- Increased effectiveness of targeted attacks against known framework or runtime vulnerabilities
- Easier exploit development based on technology stack fingerprinting

This information may be leveraged by an attacker to plan further attacks against the application and its supporting infrastructure.

4. Remediation & References

Remediation

- Disable debug mode in the production environment:

DEBUG = False

- Implement generic error handling for unexpected exceptions
- Avoid exposing stack traces, framework versions, runtime versions, and internal code references to end users
- Validate and sanitize user-controlled parameters such as next
- Log detailed server-side errors securely and present only a generic failure message to users
- Review application routes and redirect logic to ensure malformed inputs do not trigger verbose exceptions

References

- CWE-209: https://cwe.mitre.org/data/definitions/209.html
- OWASP Error Handling: https://owasp.org/www-community/Improper_Error_Handling
- Django Security Documentation: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V10 500 error through the INternal Path and Python version.png

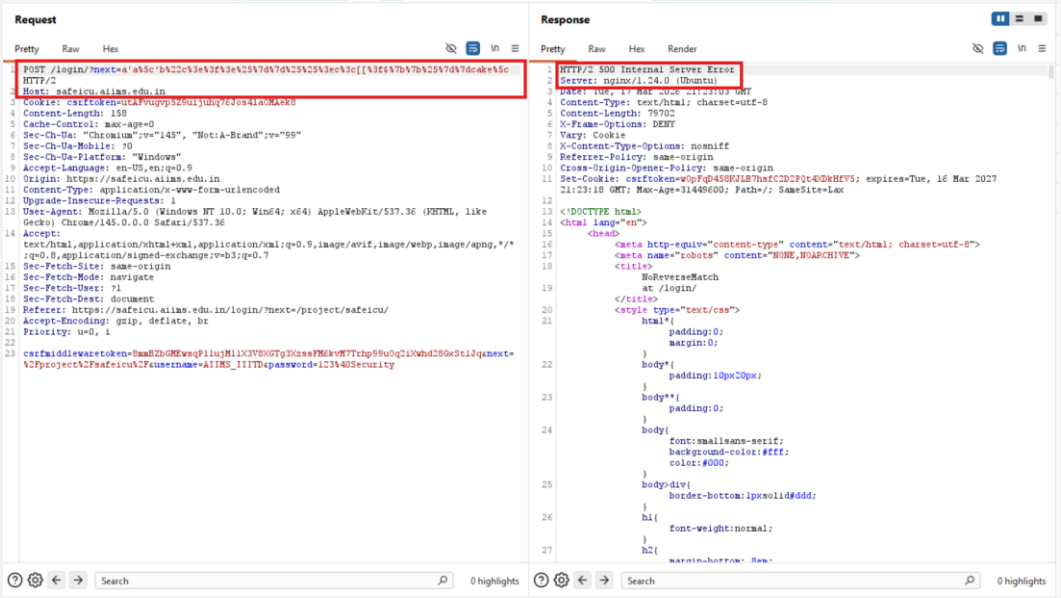
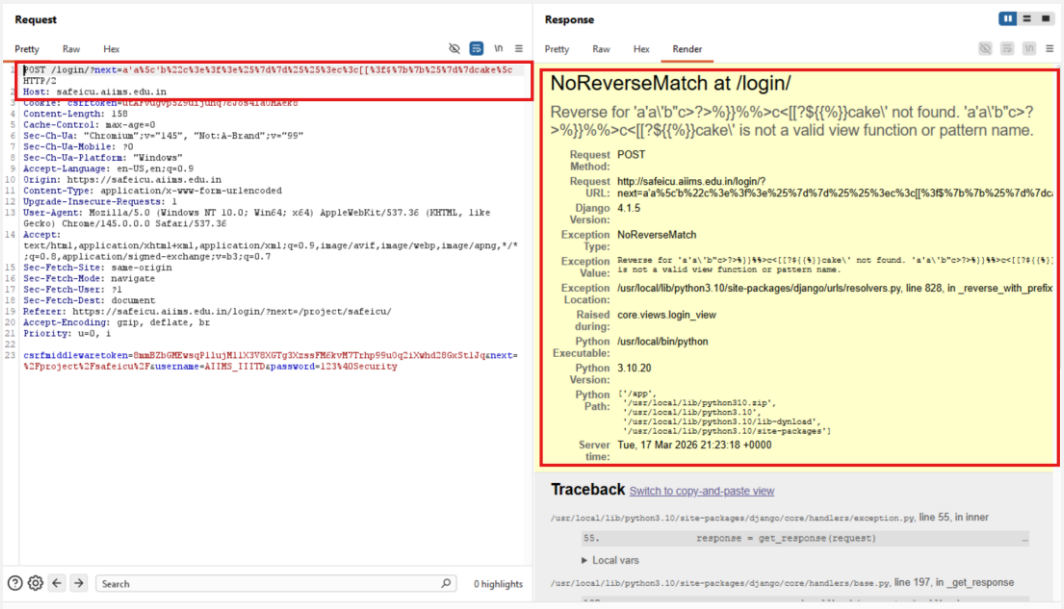


Figure: V10.1 error through the INternal Path and Python version.png


Request
Pretty
Raw
Hex
POST /login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f%7b%7b%25%7d%7dcake%5c
HTTP/2
Host: safeicu.aiims.edu.in
Content-Length: 158
Cache-Control: max-age=0
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://safeicu.aiims.edu.in
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://safeicu.aiims.edu.in/login/?next=/project/safeicu/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
csrfmiddlewaretoken=8mmBZbGMEwsqPllujM1lX3V8XGTg3XzssFM6kvM7Trhp99u0q2iXWhd28GxStlJq&next=%2Fproject%2Fsafeicu%2F&username=AIIMS_IIITD&password=123%40Security
Search
0 highlights
Response
Pretty
Raw
Hex
Render
NoReverseMatch at /login/
Reverse for 'a'a\"b"c>?>%}}%%>c<[[?${{%}}cake\' not found. 'a'a\"b"c>?>%}}%%>c<[[?${{%}}cake\' is not a valid view function or pattern name.
Request Method: POST
Request URL: http://safeicu.aiims.edu.in/login/?next=a'a%5c'b%22c%3e%3f%3e%25%7d%7d%25%25%3ec%3c[[%3f%7b%7b%25%7d%7dc
Django Version: 4.1.5
Exception Type: NoReverseMatch
Exception Value: Reverse for 'a'a\"b"c>?>%}}%%>c<[[?${{%}}cake\' not found. 'a'a\"b"c>?>%}}%%>c<[[?${{%}}cake\' is not a valid view function or pattern name.
Exception Location: /usr/local/lib/python3.10/site-packages/django/urls/resolvers.py, line 828, in _reverse_with_prefix
Raised during: core.views.login_view
Python Executable: /usr/local/bin/python
Python Version: 3.10.20
Python Path: ['/app',
'/usr/local/lib/python310.zip',
'/usr/local/lib/python3.10',
'/usr/local/lib/python3.10/lib-dynload',
'/usr/local/lib/python3.10/site-packages']
Server time: Tue, 17 Mar 2026 21:23:18 +0000
Traceback
Switch to copy-and-paste view
/usr/local/lib/python3.10/site-packages/django/core/handlers/exception.py, line 55, in inner
55. response = get_response(request)
▶ Local vars
/usr/local/lib/python3.10/site-packages/django/core/handlers/base.py, line 197, in _get_response

VULNERABILITY 11

Finding 11 — Outdated Django Version (4.1.5) with Known Security Vulnerabilities

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in
Vulnerability Title	Outdated Django Version (4.1.5) with Known Security Vulnerabilities
Severity	**Medium**
CWE	CWE-1104 – Use of Unmaintained Third Party Components
Reference	https://cwe.mitre.org/data/definitions/1104.html

2. Detailed Observation / Proof of Concept

During testing, verbose error messages revealed the Django framework version used by the application.

Observed:

- Django Version: 4.1.5

This version is outdated and has known publicly disclosed vulnerabilities.

Affected CVEs:

- CVE-2023-23969 – Denial of Service via multipart form data parsing
- CVE-2023-24580 – Denial of Service via URL parsing
- CVE-2023-31047 – Potential Cross-Site Scripting (XSS)
- CVE-2023-36053 – Open Redirect vulnerability

Screenshot Reference:

- V10 / V9 debug screenshots (version disclosure)

Technical Explanation:

The application is running an outdated version of Django that contains known vulnerabilities. These issues have been patched in later releases, but remain exploitable in this version depending on application configuration and usage.

Version disclosure (via debug errors) further amplifies this risk by enabling attackers to precisely target known exploits.

3. Impact

Use of outdated framework versions exposes the application to known and documented vulnerabilities.

Potential risks include:

- Denial of Service (DoS) attacks
- Cross-Site Scripting (XSS) attacks (admin context)
- Open Redirect attacks leading to phishing
- Increased attack surface due to known exploit availability

When combined with other vulnerabilities (such as DEBUG enabled), exploitation becomes significantly easier.

4. Remediation & References

Remediation:

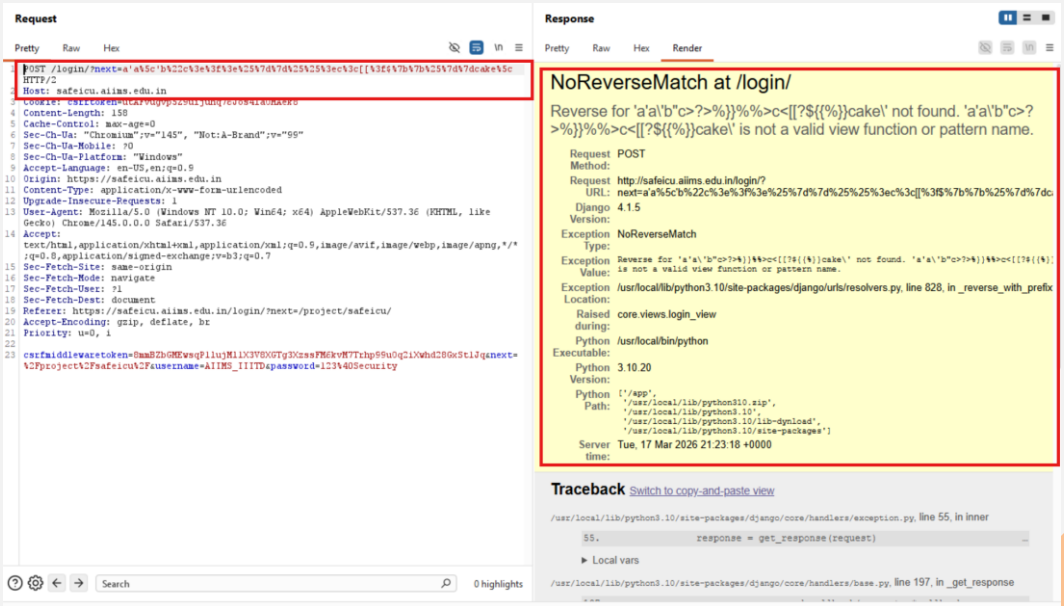
- Upgrade Django to the latest stable version:
  Recommended: Django 4.2.x (LTS) or latest available version
- Regularly monitor and apply security patches
- Implement dependency management and vulnerability scanning (e.g., pip-audit, Snyk)
- Avoid exposing version details in application responses

References:

- Django Security Releases: https://www.djangoproject.com/weblog/security/
- CVE Details: https://cve.mitre.org/
- CWE-1104: https://cwe.mitre.org/data/definitions/1104.html
- OWASP A06: https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/

Evidence / Screenshots

Figure: V11 Django outdated.png



VULNERABILITY 12

Finding 12 — Outdated nginx Version (1.24.0) with Known Security Risks

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in
Vulnerability Title	Outdated nginx Version (1.24.0) with Known Security Risks
Severity	**Medium**
CWE	CWE-1104 – Use of Unmaintained Third Party Components
Reference	https://cwe.mitre.org/data/definitions/1104.html

2. Detailed Observation / Proof of Concept

During testing, the server response headers disclosed the web server version:

Server: nginx/1.24.0 (Ubuntu)

This version is not the latest release and may be affected by known vulnerabilities addressed in newer versions.

Identified Risks / Associated CVEs

- CVE-2023-44487 – HTTP/2 Rapid Reset Attack (Denial of Service)
- CVE-2021-23017 – Heap Buffer Overflow in DNS Resolver
- CVE-2019-20372 – Improper HTTP request handling

Screenshot Reference

- V10 / response header screenshot

Technical Explanation

The server reveals its exact nginx version through HTTP response headers. This allows attackers to identify potential vulnerabilities associated with that version.

While exploitation depends on server configuration (e.g., HTTP/2 enabled, resolver usage), version disclosure combined with outdated software increases the attack surface.

3. Impact

Use of outdated nginx versions may expose the application to known vulnerabilities.

Potential risks include:

- Denial of Service (DoS) attacks via HTTP/2 mechanisms
- Memory corruption vulnerabilities (under specific configurations)

- Increased attack surface due to version fingerprinting
- Targeted exploitation using publicly available CVEs

When combined with version disclosure, attackers can efficiently map exploits to the system.

4. Remediation & References

Remediation

- Upgrade nginx to the latest stable version:

  Recommended: Latest stable or patched release (1.24.x latest or newer mainline)
- Disable server version disclosure:
  server_tokens off;
- Regularly apply security updates and patches
- Review nginx configuration (disable unnecessary modules, secure resolver settings)

References

- nginx Security Advisories: https://nginx.org/en/security_advisories.html
- CVE Database: https://cve.mitre.org/
- CWE-1104: https://cwe.mitre.org/data/definitions/1104.html
- OWASP A06: https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/

Evidence / Screenshots

Figure: V12-outdated Version ngnix Ubuntu.png



VULNERABILITY 13

Finding 13 — Missing Secure Flag on Session Cookies

1. Vulnerable Asset Detail

FIELD	VALUE
Affected Asset	https://safeicu.aiims.edu.in/login/
Vulnerability Title	Missing Secure Flag on Session Cookies
Severity	**Medium**
CWE	CWE-614 – Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
Reference	https://cwe.mitre.org/data/definitions/614.html

2. Detailed Observation / Proof of Concept

During authentication testing, the application issued session-related cookies in the HTTP response:

Request

POST /login/?next=/project/safeicu/ HTTP/2

Host: safeicu.aiims.edu.in

Response

HTTP/2 302 Found

Set-Cookie: csrftoken=MaUOXomRTUVXXFGaLFlwpqvyJKNySKdI; Path=/; SameSite=Lax

Set-Cookie: sessionid=38ntqu82y8coog3davjpk81i5uv2jqhh; HttpOnly; Path=/; SameSite=Lax

Observation

The following cookies were issued without the Secure flag:

- csrftoken
- sessionid

Although the application is served over HTTPS, the absence of the Secure attribute means these cookies may also be transmitted over unencrypted HTTP connections.

3. Impact

Cookies without the Secure flag are vulnerable to interception under certain conditions.

Potential risks include:

- Exposure of session tokens over unencrypted HTTP connections
- Session hijacking via network sniffing (e.g., public Wi-Fi)

- Increased risk of man-in-the-middle (MITM) attacks
- Unauthorized account access if session tokens are compromised

Even if the application primarily uses HTTPS, attackers may exploit edge cases such as forced HTTP requests.

4. Remediation & References

Remediation

- Ensure all sensitive cookies are set with the Secure flag:
  Set-Cookie: sessionid=...; Secure

- In Django, configure:
  SESSION_COOKIE_SECURE = True
  CSRF_COOKIE_SECURE = True
- Enforce HTTPS across the application (HSTS recommended)
- Review all cookies to ensure proper security attributes:
- Secure
- HttpOnly
- SameSite

References

- CWE-614: https://cwe.mitre.org/data/definitions/614.html
- OWASP Secure Cookie Attribute: https://owasp.org/www-community/controls/SecureCookieAttribute
- Django Security Settings: https://docs.djangoproject.com/en/stable/topics/security/

Evidence / Screenshots

Figure: V13-Secure cookie missing.png

